

NAME

dbmerge - combine per-volume sumtag databases into one

SYNOPSIS

dbmerge --database *TARGET* [-n, --dry-run] [--progress] [--allow-mixed] *SOURCE*...

DESCRIPTION

dbmerge folds any number of per-volume **sumtag**(1) databases into one *TARGET*, so that cross-volume **dedupe**(1) and **grouper**(1) runs have the single database they consume. The sumtag schema was always multi-volume (the mountpoints table exists so one database can describe files across many filesystems); one database per volume is an operational choice that buys parallel scanning, SQLite being a single-writer store. **dbmerge** closes the loop: the per-volume databases stay the scanning targets, and the merged database is a rebuildable analysis artifact, re-merged whenever the sources move on (after a rescan, or after **sumtag --prune-all** reconciles a source).

Each *SOURCE* is opened read-only, always, and is never modified. It is an experimental companion to **sumtag**(1) and never computes a checksum: only the mountpoints and files tables flow, always from source to target.

REPLACE PER MOUNTPOINT

For each mountpoint present in a source, the target's existing rows for that mountpoint are deleted and the source's rows inserted fresh. A row-level upsert would carry updates over but never remove target rows whose source rows were pruned -- the merged database would accumulate ghost rows forever. Replacement makes the merge idempotent and each source authoritative for its own mountpoints, including an empty one: a source mountpoint with zero rows still clears the target's rows for it. Target mountpoints no source mentions are left untouched, so one volume can be re-merged without feeding all of them.

The corollary is the collision refusal: the same mountpoint path recorded in two sources is an error, because the second source's replacement would delete the first's just-merged rows. Sources must partition by mountpoint -- exactly what one database per volume produces. This doubles as the guard against silently interleaving two different filesystems recorded under one mount path.

WHAT IS NEVER COPIED

grouper(1)'s derived tables (dirs, dir_files, dir_pairs, groups, group_dirs, grouper_meta) are never copied -- dir_files references files.rowid, which the merge rennumbers -- and any such artifacts already in the target are dropped, with an announcement: a merge that changed the files table has invalidated them. Rerun **grouper --prep** on the merged database; **grouper --clean-db** reclaims the space if it matters. The prescan_summary row is neither copied nor touched: it describes a filesystem walk, which the merge does not invalidate.

SAFETY CHECKS

All before the target is opened for writing:

- ✎ every source must exist and carry the sumtag tables;
- ✎ the target may not also be a source, and no source may be given twice (checked after realpath resolution, so symbolic links cannot disguise an overlap);
- ✎ the same mountpoint path in two sources is refused (see above);
- ✎ a merged corpus spanning more than one digest algorithm -- across the sources plus whatever target rows survive the merge -- is refused without **--allow-mixed**: digests only compare within one algorithm, so cross-algorithm duplicates silently fail to match.

OPTIONS

--database *TARGET*

The merged database to write. Created if missing (but not under **--dry-run**). Existing mountpoints not named by any source are left untouched. Required.

-n, --dry-run

Report what would be merged -- per-mountpoint **would merge** lines with row counts, **would replace, would drop** -- without writing anything. The target is opened read-only if it exists and is not created if it does not.

--progress

Show one aggregate progress bar on stderr covering all sources: rows-merged/rows-total with rate, percent, elapsed, and ETA. The denominator is counted up front (one cheap COUNT per source). Follows sumtag's conventions: opt-in, appears once the run has taken 2 seconds, redrawn in place, cleared on completion, suppressed when stderr is not a terminal.

--allow-mixed

Proceed even when the merged corpus would span more than one digest algorithm.

OUTPUT

One announcement per mountpoint before its rows stream across (**merge** path (*N* rows from *SOURCE*)), then the house summary block: **merge:** (rows, mountpoints, sources -- the headline prints even at zero), **replace:** and **drop:** when nonzero, then **database:** and **sources:.** Under **--dry-run** every verb gains **would**.

EXIT STATUS

- 0** Nothing to do: no rows merged, none replaced. In practice only empty sources, since replacement always rewrites.
- 1** The target was (or, under **--dry-run**, would be) modified -- the normal successful merge.
- 2** Errors, or a safety refusal.
- 130** Interrupted (Ctrl-C). Commits are per mountpoint, so an interrupted run keeps the mountpoints it completed; the summary printed reflects only committed work.

EXAMPLES

Merge three per-volume databases into one analysis database:

```
dbmerge --database merged.sqlite tank1.sqlite tank2.sqlite tank3.sqlite
```

Preview a re-merge of one volume after it was rescanned:

```
dbmerge --database merged.sqlite -n tank1.sqlite
```

Merge a whole shelf of enclosures with a progress bar:

```
dbmerge --database merged.sqlite --progress tank*.sqlite
```

SEE ALSO

sumtag(1), **grouper(1)**, **dedupe(1)**